

day01_raw_llm

练习一：手写原生 LLM 调用（不使用任何 SDK）

1. 理解 Chat Completions 接口的 HTTP 请求结构；

HTTP请求

Python |

```
1 resp = requests.post(url, headers=headers, json=payload, timeout=60)
```

url, headers, payload

Python |

```
1 LLM_BASE_URL=https://dashscope.aliyuncs.com/compatible-mode/v1
2 LLM_API_KEY=sk-ws-H.EEMLYYM.NZEe.MEUCIQD0ChL4IzsCoMg0Uf5c_nJb9iIo-1GQ2vDF5
  vdBONnd_wIgEtImVUniGwtg7cn0Q5Fo0TQBX4JKYPw8NgLQPeTx540
3 LLM_MODEL=qwen-plus
4
5 BASE_URL = os.getenv("LLM_BASE_URL", "").rstrip("/")
6 API_KEY = os.getenv("LLM_API_KEY", "")
7 MODEL = os.getenv("LLM_MODEL", "")
8
9 url = f"{BASE_URL}/chat/completions"
10
11 headers = {
12     "Authorization": f"Bearer {API_KEY}",
13     "Content-Type": "application/json",
14 }
15
16 payload = {
17     "model": MODEL,
18     "messages": messages,
19     "temperature": temperature,    # 0 → 稳定保守; 1+ → 发散有创意
20 }
```

2. 理解 messages 中的角色：system / user / assistant 各自的作用；

▼ messages 中的角色的作用

▼ system+user

Python |

```
1 messages = [  
2     {"role": "system", "content": "你是一名专业的中文客服助手，回答简洁、  
    准确、礼貌。"},  
3     {"role": "user", "content": "你好，请用一句话介绍你自己。"},  
4 ]
```

```
>>> 即将发送的请求体 messages:  
[  
  {  
    "role": "system",  
    "content": "你是一名专业的中文客服助手，回答简洁、准确、礼貌。"  
  },  
  {  
    "role": "user",  
    "content": "你好，请用一句话介绍你自己。"  
  }  
]  
  
>>> 模型回复:  
你好！我是你的智能客服助手，随时为你提供高效、准确、友好的帮助。  
  
Process finished with exit code 0
```

▼ user

Python |

```
1 messages = [  
2     # {"role": "system", "content": "你是一名专业的中文客服助手，回答简  
    洁、准确、礼貌。"},  
3     {"role": "user", "content": "你好，请用一句话介绍你自己。"},  
4 ]
```

```
>>> 即将发送的请求体 messages:  
[  
  {  
    "role": "user",  
    "content": "你好，请用一句话介绍你自己。"  
  }  
]  
  
>>> 模型回复:  
你好！我是通义千问（Qwen），阿里巴巴集团旗下的超大规模语言模型，能够回答问题、创作文字（如写故事、公文、邮件、剧本等）、进行逻辑推理、编程，还能表达观点，甚至玩游戏，支持多种语言，致力于为你提供高效、智能的交互体验！
```

```
1 ▼ messages = [  
2     {"role": "system", "content": "你是一名专业的中文客服助手，回答简洁、  
   准确、礼貌。"},  
3     {"role": "user", "content": "你好，请用一句话介绍你自己。"},  
4 ]  
5  
6 print(">>> 即将发送的请求体 messages: ")  
7 print(json.dumps(messages, ensure_ascii=False, indent=2))  
8 print("\n>>> 模型回复: ")  
9 print(chat(messages))  
10  
11 messages.append({"role": "assistant", "content": "..."})  
12 messages.append({"role": "user", "content": "帮我实现一个计算两数之差的  
   绝对值的Python函数"})  
13 print(">>> 即将发送的请求体 messages: ")  
14 print(json.dumps(messages, ensure_ascii=False, indent=2))  
15 print("\n>>> 模型回复: ")  
16 print(chat(messages))  
17
```

```

{
  "role": "system",
  "content": "你是一名专业的中文客服助手，回答简洁、准确、礼貌。"
},
{
  "role": "user",
  "content": "你好，请用一句话介绍你自己。"
}
]

>>> 模型回复：
你好！我是你的智能客服助手，随时为你提供高效、准确、友好的帮助。
>>> 即将发送的请求体 messages:
[
  {
    "role": "system",
    "content": "你是一名专业的中文客服助手，回答简洁、准确、礼貌。"
  },
  {
    "role": "user",
    "content": "你好，请用一句话介绍你自己。"
  },
  {
    "role": "assistant",
    "content": "..."
  },
  {
    "role": "user",
    "content": "帮我实现一个计算两数之差的绝对值的Python函数"
  }
]

>>> 模型回复：
```python
def abs_diff(a, b):
 return abs(a - b)
```

```

3. 思考题

- 请求体里 model / messages / temperature 分别控制什么？

回答：model决定你提问的是哪个大模型；messages决定你让大模型承担什么角色，你要大模型帮你解决的问题；temperature控制大模型针对你的提问进行思考的时候的发散程度，回答结果的发散程度，越大发散程度越大，越有创意，取值范围在0到1之间。

- system 和 user 角色有什么本质区别？

user角色指的是我们使用大模型的人，这个角色附带的content是我们对大模型说的话；system指的是大模型，content一般都是对大模型的人格底色的设定，或者是对它进行的一些要求。

- 如果去掉 system，结果会有什么变化？动手试试。

如果去掉system，大模型就会恢复默认状态，没有明确承担的角色，性格底色，语言风格等内容。

练习二：强制结构化输出（JSON）

让模型稳定返回 JSON，而不是"看心情"的自由文本。

为什么这是 Agent 的地基？

Agent 的本质是"模型决定下一步做什么"。模型要能被程序解析——它必须回答"我要调用哪个工具、参数是什么"，而这些必须是结构化的。不会结构化输出，就做不出 Function Calling，更做不出 Agent。

两种做法

1. Prompt 约束：在提示词里明确要求输出 JSON（通用，所有模型都支持）；
2. response_format={"type": "json_object"}：若模型支持，稳定性更高。

生产铁律

无论用哪种，都要加 解析失败兜底。

模型一定会有不听话的时候（这也是后面"错误处理 + 重试"的雏形）。

Prompt约束+response_format

```

1  import os
2  import json
3
4  import requests
5  from dotenv import load_dotenv
6
7  load_dotenv()
8
9  BASE_URL = os.getenv("LLM_BASE_URL", "").rstrip("/")
10 API_KEY = os.getenv("LLM_API_KEY", "")
11 MODEL = os.getenv("LLM_MODEL", "")
12
13
14 def _require_config():
15     """配置自检：把 MissingSchema 这类‘看不懂的报错’变成人话。
16
17     根因几乎总是 .env 缺失或 Key 未填写。这是一种工程习惯：
18     在昂贵/易错的调用之前，先做廉价的前置校验。
19     """
20     if not BASE_URL or not API_KEY or not MODEL or API_KEY == "your_api_k
        ey_here":
21         missing = [
22             name for name, val in
23             [("LLM_BASE_URL", BASE_URL), ("LLM_API_KEY", API_KEY), ("LLM_
        MODEL", MODEL)]
24             if not val or val == "your_api_key_here"
25         ]
26         raise SystemExit(
27             f"\n[配置错误] 缺失或未修改: {'', '.join(missing)}\n"
28             f"当前 BASE_URL = {BASE_URL!r}\n\n"
29             f"排查: 1) 本目录是否有 .env (非 .env.example) ? 没有就 cp .env.ex
        ample .env; "
30             f"2) 把 your_api_key_here 换成真实 Key; 3) .env 需与本脚本同目录。
        \n"
31         )
32
33
34 def chat(messages: list[dict], temperature: float = 0.0, json_mode: bool
    = False) -> str:
35     """对话调用。json_mode=True 时尝试启用模型的 JSON 输出模式。
36
37     注意 temperature 默认设为 0: 结构化任务要稳定，不要创造性。
38     """
39     _require_config()
40     url = f"{BASE_URL}/chat/completions"

```

```

41     headers = {
42         "Authorization": f"Bearer {API_KEY}",
43         "Content-Type": "application/json",
44     }
45     payload = {
46         "model": MODEL,
47         "messages": messages,
48         "temperature": temperature,
49     }
50     if json_mode:
51         # 并非所有模型/接口都支持, 用 try 兜底或按需去掉
52         payload["response_format"] = {"type": "json_object"}
53
54     resp = requests.post(url, headers=headers, json=payload, timeout=60)
55     resp.raise_for_status()
56     return resp.json()["choices"][0]["message"]["content"]
57
58
59 def parse_json_safely(text: str) -> dict | None:
60     """安全解析 JSON: 处理模型偶尔包裹 ```json 代码块的情况。
61
62     生产环境请务必保留这层兜底, 而不是直接 json.loads()。
63     """
64     text = text.strip()
65     if text.startswith("`"): # 去掉 ```json ... ``
66
67         text = text.strip("`")
68         if text.startswith("json"):
69             text = text[4:]
70         text = text.strip()
71     try:
72         return json.loads(text)
73     except json.JSONDecodeError:
74         return None
75
76 # — 场景: 客服 Agent 的第一步是"理解用户意图" —
77 # 我们要模型输出结构化的意图判断, 程序才能据此决定后续动作。
78 SYSTEM_PROMPT = """你是一名客服意图识别引擎。请分析用户的话, 只输出 JSON, 不要任何
79 多余文字。
80 JSON 格式如下:
81 {
82     "intent": "咨询价格 | 查询订单 | 申请退款 | 投诉建议 | 其他",
83     "emotion": "平静 | 不满 | 愤怒",
84     "need_human": true 或 false,
85     "confidence": 0 到 1 之间的小数
86 }
87 其中 need_human 表示是否需要转接人工客服 (例如用户强烈不满或问题超出范围)。"""

```

```

87
88
89 if __name__ == "__main__":
90     user_input = "我昨天买的东西到现在还没发货！你们到底怎么回事？？"
91
92     messages = [
93         {"role": "system", "content": SYSTEM_PROMPT},
94         {"role": "user", "content": user_input},
95     ]
96
97     print(f">>> 用户输入: {user_input}\n")
98
99     raw = chat(messages, temperature=0.0, json_mode=True)
100     print(">>> 模型原始输出: ")
101     print(raw)
102
103     result = parse_json_safely(raw)
104     print("\n>>> 解析结果: ")
105     if result is None:
106         # 兜底：解析失败时的降级策略（这里简化为转人工）
107         print("解析失败！降级策略：转人工处理。")
108     else:
109         print(json.dumps(result, ensure_ascii=False, indent=2))
110         if result.get("need_human"):
111             print("\n[决策] 该用户需要转人工客服。")
112

```



```
# — 场景：客服 Agent 的第一步是"理解用户意图" —
```

```
# 我们要模型输出结构化的意图判断，程序才能据此决定后续动作。
```

```
SYSTEM_PROMPT = """你是一名客服意图识别引擎。请分析用户的话，只输出 JSON，不要任何多余文字。"""
```

 02_structured_output ×

⋮

```
C:\Users\LEGION\.local\bin\uv.exe run D:/WorkBuddySpace/Claw/agent-learning/.venv/Sc
```

```
>>> 用户输入：你们这个东西怎么这么垃圾啊？？
```

```
>>> 模型原始输出：
```

```
{"intent": "投诉", "sentiment": "负面"}
```

```
>>> 解析结果：
```

```
{  
  "intent": "投诉",  
  "sentiment": "负面"  
}
```

```
Process finished with exit code 0
```

去掉所有json约束+json解析

```
# 我们要模型输出结构化的意图判断，程序才能据此决定后续动作。
SYSTEM_PROMPT = """你是一名客服意图识别引擎。请分析用户的话，只输出 JSON，不要任何多余文字。"""
```

```
if __name__ == "__main__":
    user_input = "你们这个东西怎么这么垃圾啊？"

    messages = [
        {"role": "system", "content": SYSTEM_PROMPT},
        {"role": "user", "content": user_input},
    ]

    print(f">>> 用户输入: {user_input}\n")

    raw = chat(messages, temperature=0.0, json_mode=False)
    print(">>> 模型原始输出: ")
    print(raw)

    result = raw
    # result = parse_json_safely(raw)
    print("\n>>> 解析结果: ")
    if result is None:
        # 兜底：解析失败时的降级策略（这里简化为转人工）
```

02_structured_output x

■ | :

```
C:\Users\LEGION\.local\bin\uv.exe run D:/WorkBuddySpace/Claw/agent-learning/.venv/Scripts/python.exe D:\Work
>>> 用户输入: 你们这个东西怎么这么垃圾啊？

>>> 模型原始输出:
{"intent": "投诉", "sentiment": "负面"}

>>> 解析结果:
"{\"intent\": \"投诉\", \"sentiment\": \"负面\"}"
Traceback (most recent call last):
  File "D:\WorkBuddySpace\Claw\agent-learning\day01_raw_llm\02_structured_output.py", line 127, in <module>
    if result.get("need_human"):
    ^^^^^^^^^^^
AttributeError: 'str' object has no attribute 'get'
```

Prompt约束，没有json解析

SYSTEM_PROMPT = """你是一个客服机器人。请严格按照下面的JSON格式输出。"""

JSON 格式如下:

```
{
  "intent": "咨询价格 | 查询订单 | 申请退款 | 投诉建议 | 其他",
  "emotion": "平静 | 不满 | 愤怒",
  "need_human": true 或 false,
  "confidence": 0 到 1 之间的小数
}
```

其中 need_human 表示是否需要转接人工客服（例如用户强烈不满或问题超出范围）。"""

```
if __name__ == "__main__":
    user_input = "你们这个东西怎么这么垃圾啊??"

    messages = [
        {"role": "system", "content": SYSTEM_PROMPT},
        {"role": "user", "content": user_input},
    ]

    print(f">>> 用户输入: {user_input}\n")

    raw = chat(messages, temperature=0.0, json_mode=False)
    print(">>> 模型原始输出:")
    print(raw)

    result = raw
```

02_structured_output x

```

:
"emotion": "愤怒",
"need_human": true,
"confidence": 0.95
}

>>> 解析结果:
"{\n  \"intent\": \"投诉建议\", \n  \"emotion\": \"愤怒\", \n  \"need_human\": true, \n  \"confidence\": 0.95\n}"
Traceback (most recent call last):
  File "D:\WorkBuddySpace\Claw\agent-learning\day01_raw_llm\02_structured_output.py", line 135, in <module>
    if result.get("need_human"):
        ^^^^^^^^^^^
AttributeError: 'str' object has no attribute 'get'
```

完成后自问

- 为什么"prompt 里写了要 JSON", 模型有时还是会输出别的?
- 如果模型返回的 JSON 解析失败, 你的程序应该怎么办?